

NUTRIALIANZA S.A.

Sistema de Monitoreo Inteligente con Docker e IA

Resumen académico

ESCENARIO 1

Saturacion HTTP en Nginx

Objetivo: comprobar la detección automática de una interrupción total del servicio web, validar la indisponibilidad de los health checks HTTP/HTTPS y del puerto 443, y confirmar la recuperación del servicio después de reiniciar Nginx.

Actividad	Tarea 1 - Emulación de Saturación en Producción
Resultado general	Escenario completado comprobando la capacidad del sistema de monitoreo de NutriAlianza S.A. para detectar una saturación del servicio web Nginx
Fecha de prueba	4 de Agosto de 2026
Integrantes	Julio Gutiérrez Orozco María José Miranda López

Escenario 1 – Saturación HTTP en Nginx

Objetivo de la prueba

El primer escenario tuvo como objetivo comprobar la capacidad del sistema de monitoreo de NutriAlianza S.A. para detectar una saturación del servicio web Nginx, identificar el incremento anormal de solicitudes HTTP y generar una alerta automática con apoyo de inteligencia artificial.

Este escenario corresponde a una de las pruebas obligatorias definidas para el proyecto: provocar saturación HTTP mediante una herramienta de carga, obtener respuestas HTTP 503 y verificar que el incidente sea detectado por la infraestructura de monitoreo.

Para la prueba se utilizó ApacheBench (ab) desde WSL2, mientras Nginx, Prometheus, Nginx Exporter, N8N, Groq y Telegram operaban dentro de la arquitectura del proyecto.

Preparación del monitoreo de Nginx

Antes de ejecutar la prueba se ajustó la configuración de Nginx para incorporar tres elementos principales:

- /health, utilizado para comprobar que el servicio respondiera correctamente.
- /stub_status, utilizado por Nginx Exporter para obtener métricas internas del servidor.
- /stress, creado exclusivamente para ejecutar la prueba de saturación.

El endpoint /stress se configuró con un límite de solicitudes. Cuando la cantidad de peticiones superaba la capacidad definida, Nginx respondía con el código **HTTP 503 Service Unavailable**.

También se agregó nginx-exporter al entorno Docker. Este componente expuso las métricas de Nginx en el puerto 9113, las cuales posteriormente fueron recolectadas por Prometheus.

La conexión se verificó mediante la métrica:

nginx_up = 1 Este valor confirmó que Prometheus podía consultar correctamente el estado de Nginx.

```
nginx-exporter:
  image: nginx/nginx-exporter:1.0.0
  ports:
    - 9113:9113
  depends_on:
    - nginx
  networks:
    - nutrialianza-net
```

Configuración de la detección automática

Prometheus consultó la tasa de solicitudes HTTP mediante la expresión:

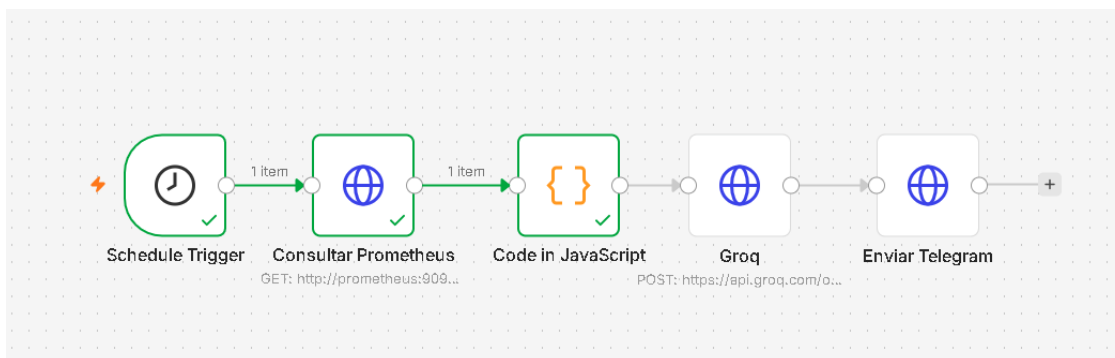
```
rate(nginx_http_requests_total[1m])
```

Esta métrica permitió calcular la cantidad aproximada de solicitudes por segundo recibidas por Nginx durante el último minuto.

En N8N se configuró un umbral de:

20 solicitudes por segundo

El nodo JavaScript evaluó el valor obtenido desde Prometheus. Si el resultado permanecía por debajo de 20 solicitudes por segundo, el flujo terminaba sin generar alertas. Cuando el valor igualaba o superaba el umbral, N8N continuaba hacia Groq y posteriormente hacia Telegram.



Para reducir el tiempo de detección durante la prueba, el Schedule Trigger de N8N se configuró para consultar Prometheus cada **10 segundos**.

La lógica final quedó definida de la siguiente manera:

Prometheus → N8N → evaluación del umbral → Groq → Telegram

Groq recibió información real del incidente, como el servicio afectado, la tasa de solicitudes detectada y el umbral configurado. A partir de estos datos generó un diagnóstico con severidad, causa probable, impacto y recomendación técnica.

Ejecución de la prueba de saturación

La prueba definitiva se ejecutó desde WSL2 mediante ApacheBench con el siguiente comando:

- `ab -n 50000 -c 50 http://localhost/stress`

La prueba envió **50.000 solicitudes HTTP** al endpoint /stress, con un nivel de concurrencia de **50 conexiones simultáneas**.

ApacheBench obtuvo los siguientes resultados:

- Solicitudes completas: **50.000**
- Solicitudes fallidas: **49.936**
- Respuestas diferentes de 2xx: **49.936**
- Solicitudes procesadas por segundo: **4.664,76 req/s**
- Tiempo total de prueba: **10,719 segundos**
- Nivel de concurrencia: **50**

De las 50.000 solicitudes ejecutadas, solamente **64 solicitudes** no fueron registradas como respuestas fallidas.

Esto representa aproximadamente:

- **99,872 % de solicitudes no exitosas**
- **0,128 % de solicitudes restantes**

El resultado demuestra que el mecanismo de limitación configurado en Nginx respondió correctamente frente a una carga considerablemente superior a su umbral permitido.

```
julio@DESKTOP-JN2R04E:~/nutritionalianza-monitoreo$ ab -n 50000 -c 50 http://localhost/stress
This is ApacheBench, Version 2.3 <$Revision: 1903618 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/
```

```
Benchmarking localhost (be patient)
Completed 5000 requests
Completed 10000 requests
Completed 15000 requests
Completed 20000 requests
Completed 25000 requests
Completed 30000 requests
Completed 35000 requests
Completed 40000 requests
Completed 45000 requests
Completed 50000 requests
Finished 50000 requests
```

```
Server Software:      nginx/1.31.3
Server Hostname:      localhost
Server Port:          80

Document Path:        /stress
Document Length:      333 bytes

Concurrency Level:    50
Time taken for tests:  10.719 seconds
Complete requests:    50000
Failed requests:      49936
    (Connect: 0, Receive: 0, Length: 49936, Exceptions: 0)
Non-2xx responses:    49936
Total transferred:    18462608 bytes
HTML transferred:    9858704 bytes
Requests per second:  4664.76 [#/sec] (mean)
Time per request:     10.719 [ms] (mean)
Time per request:     0.214 [ms] (mean, across all concurrent requests)
Transfer rate:        1682.10 [Kbytes/sec] received
```

Connection Times (ms)				
	min	mean[+/-sd]	median	max
Connect:	0	1 1.7	0	31
Processing:	0	10 6.9	8	95
Waiting:	0	9 6.6	7	95
Total:	0	11 7.0	9	96

Percentage of the requests served within a certain time (ms)	
50%	9
66%	11
75%	13
80%	14
90%	19
95%	24
98%	30
99%	36
100%	96 (longest request)

Detección de la anomalía en Prometheus

Mientras ApacheBench ejecutaba la carga, una segunda consola consultó continuamente Prometheus.

Durante el punto de mayor actividad se observó aproximadamente:

629,59 solicitudes por segundo

Este valor superó ampliamente el umbral configurado en N8N:

629,59 RPS > 20 RPS

Prometheus, por lo tanto, identificó de forma clara el cambio entre el comportamiento normal del servidor y el periodo de saturación.

El valor mostrado por Prometheus no tiene que coincidir exactamente con los **4.664,76 req/s** calculados por ApacheBench. ApacheBench obtiene el promedio de solicitudes procesadas durante toda su ejecución, mientras la consulta de Prometheus utilizada en el proyecto calcula una tasa sobre una ventana de un minuto mediante `rate(...[1m])`.

Evidencia de respuestas HTTP 503

Después de finalizar la prueba se consultó directamente el archivo `access.log` de Nginx.

Los registros mostraron entradas como:

```
GET /stress HTTP/1.0" 503
```

Esto permitió confirmar que las respuestas no exitosas reportadas por ApacheBench correspondían específicamente a errores **HTTP 503 Service Unavailable**.

[illegible]

Además, el conteo acumulado de respuestas 503 después de las diferentes pruebas realizadas durante la validación fue:

149.816 respuestas HTTP 503

```
julio@DESKTOP-JN2R04E:~/nutrialianza-monitoreo$ grep -c ' 503 ' logs/nginx/access.log
149816
```

El número coincide con la suma de los errores obtenidos en las tres ejecuciones realizadas:

- Primera prueba: **49.934** Failed requests: 49934
- Segunda prueba: **49.946** Failed requests: 49946
- Prueba definitiva: **49.936** Failed requests: 49936
- Total: **149.816**

La prueba definitiva es la que se utiliza como evidencia principal del escenario; las ejecuciones anteriores permitieron ajustar el intervalo de detección de N8N y validar el comportamiento del sistema.

Activación de N8N, Groq y Telegram

Durante la prueba definitiva, N8N consultó Prometheus cada 10 segundos. Al detectar un valor superior al umbral de 20 solicitudes por segundo, el nodo JavaScript permitió que el flujo continuara.

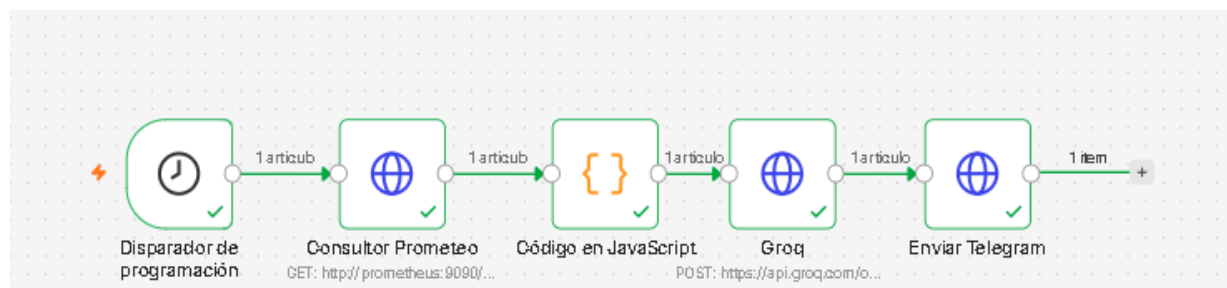
A partir de ese momento se ejecutó la siguiente secuencia:

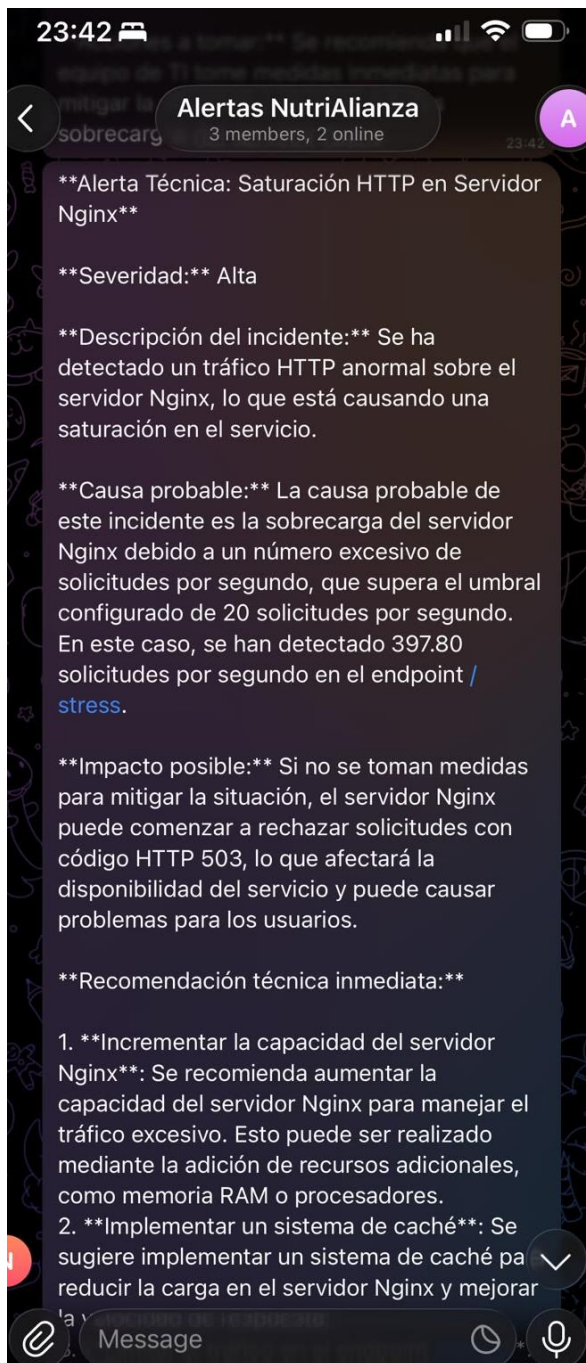
N8N → Groq → Telegram

Groq recibió los datos del incidente y generó un análisis técnico en español. El mensaje incluyó información sobre la severidad de la saturación, la posible causa, el impacto sobre el servicio y una recomendación para el equipo de TI.

Finalmente, N8N envió de forma automática el contenido generado por Groq al bot de Telegram configurado para NutriAlianza S.A.

La recepción del mensaje confirmó que el sistema no solamente detectó el incremento de tráfico, sino que completó correctamente toda la cadena de análisis y notificación.





Resultado del escenario

El escenario permitió validar de forma completa el funcionamiento del sistema ante una condición real de saturación HTTP.

La prueba confirmó los siguientes puntos:

- ApacheBench generó una carga de **50.000 solicitudes HTTP**.
- Nginx alcanzó una tasa aproximada de **4.664,76 solicitudes por segundo** durante la ejecución de ApacheBench.

- Se registraron **49.936 respuestas no exitosas** en la prueba definitiva.
- Los registros de Nginx confirmaron que las respuestas correspondían a **HTTP 503**.
- Prometheus detectó aproximadamente **629,59 solicitudes por segundo**, muy por encima del umbral de **20 RPS**.
- N8N identificó automáticamente la condición de saturación.
- Groq generó el análisis técnico del incidente.
- Telegram recibió correctamente la alerta automática.

Por lo tanto, el **Escenario 1 – Saturación HTTP en Nginx se considera completado satisfactoriamente**, ya que se demostró la generación del incidente, su registro, detección, análisis mediante inteligencia artificial y notificación automática al equipo de soporte.

Escenario 2 – Saturación HTTP en Nginx

Objetivo de la prueba

Copias de seguridad: